

WORKED EXAMPLE

Start with root mean square

RMS means square each value, average the squares, then take the square root.

$$\text{RMS}(a_1, \dots, a_n) = \sqrt{\frac{a_1^2 + \dots + a_n^2}{n}}$$

Squaring removes signs, so large negative and positive movements both contribute. Averaging gives a mean square. The final root returns to the units of the original quantity.

Lecture 5 page 114 lists coordinate movements. We first compute their ordinary RMS values, then use a different, exponentially weighted calculation for RMSProp optimizer memory.

The optimizer rescales each coordinate using its own recent squared gradients. The base learning rate still controls the overall size of the step.

WORKED EXAMPLE

RMS of the horizontal coordinate sequence

Use the five X components listed on lecture page114.

$$x = (1, 1, 2, 1, 1.5)$$

$$x^2 = (1, 1, 4, 1, 2.25)$$

$$\sum x_i^2 = 1 + 1 + 4 + 1 + 2.25 = 9.25$$

$$\text{mean square} = 9.25/5 = 1.85$$

$$\text{RMS}_x = \sqrt{1.85} \approx 1.360147$$

This is one root after averaging squares, not an average of square roots or an average of signed components.

WORKED EXAMPLE

RMS of the vertical coordinate sequence

Negative movements count positively after squaring.

$$y = (2.5, -3, 2.5, -2, 1.5)$$

$$y^2 = (6.25, 9, 6.25, 4, 2.25)$$

$$\sum y_i^2 = 6.25 + 9 + 6.25 + 4 + 2.25 = 27.75$$

$$\text{mean square} = 27.75/5 = 5.55$$

$$\text{RMS}_y = \sqrt{5.55} \approx 2.355844$$

The larger vertical RMS suggests more scaling down in that coordinate. This movement illustration motivates the optimizer; it is not its actual recursive state.

WORKED EXAMPLE

RMS is different from standard deviation

Variance first subtracts the mean; mean square does not.

$$a = (2, 2), \quad \bar{a} = 2$$

$$\text{RMS} = \sqrt{(2^2 + 2^2)/2} = 2$$

$$\text{variance} = ((2 - 2)^2 + (2 - 2)^2)/2 = 0$$

$$\text{standard deviation} = \sqrt{0} = 0$$

The constant sequence has nonzero magnitude but no variation around its mean. RMS measures the former; standard deviation measures the latter.

A squared gradient is also not a second derivative or a Hessian entry. Square the numerical first derivative already computed by backpropagation.

WORKED EXAMPLE

The course RMSProp recurrence

Lecture 5 page 116 places epsilon inside the square root.

$$s_{t,j} = \gamma s_{t-1,j} + (1 - \gamma) g_{t,j}^2$$

$$w_{t,j} = w_{t-1,j} - \frac{\eta g_{t,j}}{\sqrt{s_{t,j} + \epsilon}}$$

t counts updates and j identifies a parameter coordinate. g is the current averaged batch gradient. s stores an exponentially weighted history of its squares. Every multiplication and square is coordinatewise.

Gamma retains old memory; $1 - \gamma$ weights the new squared gradient. Eta is the base rate. Epsilon is a positive stabilizer in implementations.

Initialize s once at the start of the training run and retain it across updates. The zero-start estimate is not an unbiased second moment.

WORKED EXAMPLE

Average the current batch before squaring

Two operations that look similar give different numbers.

example gradients = $(2, -2)$

$$g = (2 + (-2))/2 = 0 \implies g^2 = 0$$

$$\text{mean of squares} = (2^2 + (-2)^2)/2 = 4$$

For the lecture mini-batch algorithm, use the first result in the RMSProp recurrence. Its g is the already averaged batch gradient. Do not replace it with the mean of squared example gradients.

Across optimizer updates, keep the squared-gradient memory. Within a batch, clear the temporary sum before averaging new example gradients.

WORKED EXAMPLE

Set up the two-coordinate optimizer exercise

The gradient vectors are supplied inputs to this state calculation.

$$w_0 = (1, 1), \quad s_0 = (0, 0), \quad \gamma = 0.5, \quad \eta = 0.1$$

$$g_1 = (2, 4), \quad g_2 = (2, -4)$$

$\varepsilon = 0$ for this positive denominator example only

These gradients are not claimed to come from a fixed quadratic. Focus on the optimizer calculation after backpropagation has produced them.

Epsilon is zero only to keep these hand calculations simple; every denominator below is positive. d denotes that denominator. Keep full precision between updates; displayed values are rounded. A positive-epsilon calculation follows.

WORKED EXAMPLE

update 1, coordinate 1

Square, update memory, take the root, then subtract the signed normalized gradient.

$$g = 2, \quad g^2 = (2)^2 = 4$$

$$s_{new} = 0.5(0) + 0.5(4) = 2$$

$$d = \sqrt{2 + 0} \approx 1.414214$$

$$\text{step} = \frac{0.1(2)}{\sqrt{2}} \approx 0.141421$$

$$w_{new} \approx 1.000000 - (0.141421) = 0.858579$$

The squared state stays nonnegative. The numerator keeps the gradient sign; subtracting a negative step increases the parameter.

WORKED EXAMPLE

update 1, coordinate 2

Square, update memory, take the root, then subtract the signed normalized gradient.

$$g = 4, \quad g^2 = (4)^2 = 16$$

$$s_{new} = 0.5(0) + 0.5(16) = 8$$

$$d = \sqrt{8 + 0} \approx 2.828427$$

$$\text{step} = \frac{0.1(4)}{\sqrt{8}} \approx 0.141421$$

$$w_{new} \approx 1.000000 - (0.141421) = 0.858579$$

The squared state stays nonnegative. The numerator keeps the gradient sign; subtracting a negative step increases the parameter.

WORKED EXAMPLE

update 2, coordinate 1

Square, update memory, take the root, then subtract the signed normalized gradient.

$$g = 2, \quad g^2 = (2)^2 = 4$$

$$s_{new} = 0.5(2) + 0.5(4) = 3$$

$$d = \sqrt{3 + 0} \approx 1.732051$$

$$\text{step} = \frac{0.1(2)}{\sqrt{3}} \approx 0.115470$$

$$w_{new} \approx 0.858579 - (0.115470) = 0.743109$$

The squared state stays nonnegative. The numerator keeps the gradient sign; subtracting a negative step increases the parameter.

WORKED EXAMPLE

update 2, coordinate 2

Square, update memory, take the root, then subtract the signed normalized gradient.

$$g = -4, \quad g^2 = (-4)^2 = 16$$

$$s_{new} = 0.5(8) + 0.5(16) = 12$$

$$d = \sqrt{12 + 0} \approx 3.464102$$

$$\text{step} = \frac{0.1(-4)}{\sqrt{12}} \approx -0.115470$$

$$w_{new} \approx 0.858579 - (-0.115470) = 0.974049$$

The squared state stays nonnegative. The numerator keeps the gradient sign; subtracting a negative step increases the parameter.

WORKED EXAMPLE

Read the vector states together

Each parameter retains its own squared-gradient history.

update	g	new s	new w
1	(2,4)	(2,8)	(.858579,.858579)
2	(2,-4)	(3,12)	(.743109,.974049)

$$\eta/\sqrt{s_1} = (0.1/\sqrt{2}, 0.1/\sqrt{8})$$

$$\approx (0.070711, 0.035355)$$

Coordinate2 has twice the first gradient magnitude but half the effective rate, so the first normalized steps are equal. This is a property of these numbers, not a universal equality.

WORKED EXAMPLE

Expand the memory to see its weights

For $\gamma=.5$ and a zero initial state.

$$s_1 = 0.5g_1^2$$

$$s_2 = 0.5s_1 + 0.5g_2^2 = 0.25g_1^2 + 0.5g_2^2$$

$$\text{coordinate 2 : } 0.25(4^2) + 0.5((-4)^2) = 4 + 8 = 12$$

The latest squared gradient has weight .5; the preceding one has .25. Their weights sum to .75 because the remaining .25 multiplies the initial zero state.

This is an exponential moving average, not the uniform mean of the two squares. Opposite signs do not cancel inside this squared memory. RMSProp here has no Adam-style bias correction.

WORKED EXAMPLE

Positive epsilon changes the denominator

Use a supplied new state $s=4$, gradient $g=2$, $\eta=.1$, and $\epsilon=1$.

course : $d = \sqrt{4 + 1} = \sqrt{5} \approx 2.236068$

step = $0.1(2)/\sqrt{5} \approx 0.089443$

outside variant : $d = \sqrt{4} + 1 = 3$

step = $0.1(2)/3 \approx 0.066667$

For a current parameter1, the course update gives about .910557; the outside-root variant gives .933333. These are different formulas, so declare the convention before substituting.

WORKED EXAMPLE

Why a stabilizer matters at zero

A zero new state and zero gradient should not create a division by zero.

$$s = 0, g = 0, \varepsilon = 1: \quad \text{step} = 0.1(0)/\sqrt{0 + 1} = 0$$

$$\varepsilon = 0: \quad 0/\sqrt{0} = 0/0 \quad \text{undefined}$$

The value1 is deliberately large to make the comparison visible. It is an illustration of placement and stabilization, not a recommended practical default.

The main example safely omitted epsilon only because all its computed denominators were positive.

INDEPENDENT PRACTICE

Your turn: a fresh sign-changing coordinate

Reset both parameter and memory for this separate practice run.

$$w_0 = (0, 0), \quad s_0 = (0, 0), \quad g_1 = (1, 2), \quad g_2 = (-1, 2)$$

$$\gamma = 0.5, \quad \eta = 0.1, \quad \varepsilon = 0$$

For each update and coordinate, calculate the gradient square, new memory, denominator, signed step and new parameter. Keep the first state when processing update2.

Separately, compare the course and outside-root denominator for $s=4$, $\text{epsilon}=1$, and explain why RMS and standard deviation differ on $[2,2]$.

WORKED ANSWER

Practice: update 1, coordinate 1

Square, update memory, take the root, then subtract the signed normalized gradient.

$$g = 1, \quad g^2 = (1)^2 = 1$$

$$s_{new} = 0.5(0) + 0.5(1) = 0.5$$

$$d = \sqrt{0.5 + 0} \approx 0.707107$$

$$\text{step} = \frac{0.1(1)}{\sqrt{0.5}} \approx 0.141421$$

$$w_{new} \approx 0.000000 - (0.141421) = -0.141421$$

The squared state stays nonnegative. The numerator keeps the gradient sign; subtracting a negative step increases the parameter.

WORKED ANSWER

Practice: update 1, coordinate 2

Square, update memory, take the root, then subtract the signed normalized gradient.

$$g = 2, \quad g^2 = (2)^2 = 4$$

$$s_{new} = 0.5(0) + 0.5(4) = 2$$

$$d = \sqrt{2 + 0} \approx 1.414214$$

$$\text{step} = \frac{0.1(2)}{\sqrt{2}} \approx 0.141421$$

$$w_{new} \approx 0.000000 - (0.141421) = -0.141421$$

The squared state stays nonnegative. The numerator keeps the gradient sign; subtracting a negative step increases the parameter.

WORKED ANSWER

Practice: update 2, coordinate 1

Square, update memory, take the root, then subtract the signed normalized gradient.

$$g = -1, \quad g^2 = (-1)^2 = 1$$

$$s_{new} = 0.5(0.5) + 0.5(1) = 0.75$$

$$d = \sqrt{0.75 + 0} \approx 0.866025$$

$$\text{step} = \frac{0.1(-1)}{\sqrt{0.75}} \approx -0.115470$$

$$w_{new} \approx -0.141421 - (-0.115470) = -0.025951$$

The squared state stays nonnegative. The numerator keeps the gradient sign; subtracting a negative step increases the parameter.

WORKED ANSWER

Practice: update 2, coordinate 2

Square, update memory, take the root, then subtract the signed normalized gradient.

$$g = 2, \quad g^2 = (2)^2 = 4$$

$$s_{new} = 0.5(2) + 0.5(4) = 3$$

$$d = \sqrt{3 + 0} \approx 1.732051$$

$$\text{step} = \frac{0.1(2)}{\sqrt{3}} \approx 0.115470$$

$$w_{new} \approx -0.141421 - (0.115470) = -0.256891$$

The squared state stays nonnegative. The numerator keeps the gradient sign; subtracting a negative step increases the parameter.

WORKED ANSWER

Practice checkpoint and common mistakes

All new parameters use the state computed in the same update.

$$s_1 = (0.5, 2), \quad w_1 \approx (-0.141421, -0.141421)$$

$$s_2 = (0.75, 3), \quad w_2 \approx (-0.025951, -0.256891)$$

$$\sqrt{4 + 1} = \sqrt{5} \approx 2.236068, \quad \sqrt{4} + 1 = 3$$

For [2,2], RMS is 2 while standard deviation is 0: magnitude is different from spread around the mean.

Avoid squaring before averaging the batch, retaining a negative sign after squaring, dropping the gradient sign in the update, using the previous s in the denominator, or resetting memory between batches.